

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #10 - Semana 11 – Libro C - Capítulo 7
Entrada y salida

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 23/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 135–150.

Abstract—*C provides input and output functions through its standard library, which enables interaction with the environment. Key functions include getchar for reading input and putchar for writing output, while formatted input and output are handled by scanf and printf. File operations, such as reading and writing files, are performed using functions like fopen and fclose. Error handling is done through stderr, and memory management can be handled with malloc and free. Additional utilities include string manipulation, character conversion, and random number generation, enhancing the functionality of C programs.*

I. Entrada y salida en C

Las funciones de entrada y salida no forman parte del lenguaje C en sí, sino que se implementan a través de la biblioteca estándar. Estas funciones incluyen manejo de cadenas, gestión de almacenamiento, rutinas matemáticas y, particularmente, las funciones de entrada y salida. Las bibliotecas en C están diseñadas para ser compatibles entre sistemas, lo que permite la portabilidad de programas. Las funciones de la biblioteca están organizadas en varios encabezados como `<stdio.h>`, `<string.h>` y `<ctype.h>`. El enfoque de este capítulo está en cómo usar la entrada y salida estándar en C.

II. Entrada y salida estándar

A. Entrada estándar: getchar

La función `getchar` lee un carácter de la entrada estándar (normalmente el teclado) y devuelve el carácter leído, o EOF si llega al final del archivo. Un programa puede redirigir la entrada desde un archivo o la salida de otro programa utilizando tuberías.

Ejemplo:

```
int c;
while ((c = getchar()) != EOF)
    putchar(tolower(c));
```

En este ejemplo, `getchar` se usa para leer caracteres y `putchar` para escribirlos en la salida estándar, convirtiéndolos a minúsculas con `tolower`.

B. Salida estándar: `putchar`

`putchar` escribe un carácter en la salida estándar (normalmente la pantalla). También es posible redirigir la salida a un archivo o a otro programa mediante tuberías.

III. Salida formateada: `printf`

La función `printf` permite formatear y convertir valores internos a caracteres, imprimiéndolos en la salida estándar. La cadena de formato en `printf` contiene caracteres ordinarios que se copian directamente a la salida, junto con especificadores de conversión que controlan cómo se muestran los argumentos.

Especificadores comunes:

- `%d` e `%i`: enteros decimales
- `%s`: cadena de caracteres
- `%f`: números de punto flotante

Ejemplo:

```
int max = 10;
char *s = "hello";
printf("%.s\n", max, s); // Imprime "hello"
```

En este ejemplo, el formato `%.s` limita el número de caracteres impresos a un máximo de 10.

IV. Listas de argumentos de longitud variable

C permite crear funciones como `printf` que aceptan una lista variable de argumentos utilizando macros definidas en `<stdarg.h>`. Estas macros incluyen:

- `va_start`: inicializa la lista de argumentos.
- `va_arg`: recupera el siguiente argumento.
- `va_end`: finaliza el procesamiento de la lista de argumentos.

Ejemplo:

```
#include <stdarg.h>
void minprintf(char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    for (char *p = fmt; *p; p++) {
        if (*p != '%') {
```

```

        putchar(*p);
        continue;
    }
    switch (*++p) {
        case 'd': printf("%d", va_arg(ap, int)); break;
        case 's': printf("%s", va_arg(ap, char *)); break;
    }
}
va_end(ap);
}

```

En este ejemplo, `minprintf` procesa una lista de argumentos variables e imprime los valores según el formato especificado.

V. Entrada formateada: `scanf`

`scanf` es la contraparte de `printf` para la entrada formateada. Lee datos de la entrada estándar y los convierte según el formato especificado, almacenando los resultados en las direcciones proporcionadas como argumentos.

Ejemplo:

```

int day, year;
char month[20];
scanf("%d %s %d", &day, month, &year);

```

En este ejemplo, `scanf` lee un día, un nombre de mes y un año de la entrada.

VI. Acceso a archivos

Los programas pueden leer y escribir en archivos mediante punteros de archivo. Las funciones básicas incluyen:

- `fopen`: abre un archivo.
- `fclose`: cierra un archivo.
- `fscanf` y `fprintf`: versiones de `scanf` y `printf` que operan en archivos.

Ejemplo de un programa "cat":

```

#include <stdio.h>
void filecopy(FILE *ifp, FILE *ofp) {
    int c;
    while ((c = getc(ifp)) != EOF)
        putc(c, ofp);
}

int main(int argc, char *argv[]) {
    FILE *fp;
    if (argc == 1)
        filecopy(stdin, stdout);
    else
        while (--argc > 0) {
            if ((fp = fopen(*++argv, "r")) == NULL) {
                fprintf(stderr, "cat: can't open %s\n", *argv);
                return 1;
            }
        }
}

```

```

        filecopy(fp, stdout);
        fclose(fp);
    }
    return 0;
}

```

Este ejemplo lee archivos de entrada y los concatena en la salida estándar.

VII. Manejo de errores y flujo stderr

El flujo stderr permite manejar errores de forma separada de la salida estándar. Es útil para asegurar que los mensajes de error aparezcan en la pantalla, aunque la salida estándar se redirija a un archivo o “tubería”.

Ejemplo de manejo de errores:

```

if ((fp = fopen(filename, "r")) == NULL) {
    fprintf(stderr, "Error: no se puede abrir %s\n", filename);
    exit(1);
}

```

VIII. Entrada y salida de líneas

La biblioteca estándar proporciona funciones como fgets para leer líneas y fputs para escribir cadenas.

Ejemplo de uso de fgets:

```

char line[100];
if (fgets(line, 100, stdin) != NULL)
    printf("%s", line);

```

En este ejemplo, fgets lee una línea de la entrada estándar.

IX. Funciones misceláneas

El lenguaje C incluye muchas funciones útiles:

- **Operaciones con cadenas:** funciones como strcpy, strlen, y strcmp en <string.h>.
- **Pruebas y conversiones de caracteres:** funciones como isalpha, toupper y tolower en <ctype.h>.
- **Manejo de memoria dinámica:** malloc, calloc y free en <stdlib.h>.
- **Generación de números aleatorios:** rand y srand en <stdlib.h>.
- **Funciones matemáticas:** como sin, cos, y sqrt en <math.h>.